AI-Based Smart Logistics System

Design Document

1. Project overview

This project is an AI-powered logistics and delivery system that supports multiple delivery types and optimizes operations using intelligent algorithms. The system aims to improve delivery speed, reduce operational cost, handle different delivery scenarios efficiently, and provide real-time tracking plus smart decision-making.

2. Objectives

- Build a scalable logistics platform.
- Implement intelligent delivery routing.
- Optimize cost using order batching (order clubbing).
- Support priority and customized deliveries.
- Provide role-based system access.

3. Scope and delivery types

3.1 Quick delivery (Intracity)

- Direct delivery within the same city.
- Instant driver assignment.
- No warehouse involvement.

3.2 Warehouse delivery (Intercity)

- Parcel moves via warehouses.
- Uses batching and optimization algorithms.

Key features:

- Warehouse A → Warehouse B transfer.
- Order clubbing (batching).
- Priority vs normal delivery handling.

3.3 Customised delivery

- For fragile, valuable, or shifting items.

- Requires admin approval.
- Direct delivery via assigned agent.

4. Users and roles

- **User**
  - Create bookings
  - Track deliveries
  - Make payments
- **Driver**
  - Accept deliveries
  - Update delivery status
  - Share live location
- **Admin**
  - Approve custom deliveries
  - Assign drivers or agents
  - Manage system

5. High-level architecture

Frontend (Angular) → Backend (Spring Boot) → Services → MySQL

5.1 Components

- **Frontend (Angular)**
  - Booking creation and tracking UI
  - Role-based routes (User, Driver, Admin)
  - Integrates with backend APIs
- **Backend (Spring Boot)**
  - Spring Security + JWT authentication
  - REST APIs for booking, matching, tracking, payments
  - Service layer for business logic
- **Core services (logical modules)**
  - Authentication Service
  - Booking Service
  - Pricing Engine
  - Matching Engine
  - Tracking Service
  - Warehouse Management
  - Notification Service

5.2 External integrations

- **Google Maps API** for distance calculation and tracking.
- **Weather API** for pricing adjustments.
- **WebSockets (Socket.IO / STOMP over WebSocket)** for real-time tracking updates.
- **Email or SMS** for notifications.
- **Firebase (optional)** for push notifications.

6. Tech stack

- **Frontend:** Angular, TypeScript, HTML, SCSS, Angular Router, HTTP Client
- **Backend:** Spring Boot, Spring Security, JWT, Spring Data JPA
- **Database:** MySQL

7. Core workflows and lifecycle

7.1 Booking workflow

1. Authentication (role-based routing)
2. Booking creation (delivery type + details)
3. Pricing calculation
4. Processing
    - Quick: direct driver assignment
    - Warehouse: assign warehouses, batch orders, transport
    - Custom: admin approval, agent assigned
5. Tracking (real-time updates)
6. Completion + notification

7.2 Delivery lifecycle (status model)

`CREATED → APPROVED → ASSIGNED → PICKED → IN_TRANSIT → DELIVERED`

Additional statuses:

- `REASSIGNING`, `CANCELLED`, `FAILED`

8. Algorithms (design notes)

8.1 Driver matching

Goal: assign the best driver

- Filter drivers by availability.
- Compute distance to pickup.
- Rank by distance, rating, and vehicle type.

8.2 Best route service (route optimization)

Goal: find the best route for pricing and execution.

- Uses distance, estimated time, toll cost, and road condition signals.
- Uses weather and traffic conditions when available.
- Combines a classical shortest-path baseline (e.g., Dijkstra) with an AI model that recommends the best route.
- Outputs route options and a selected route with an explainable score (cost vs time).

8.3 Dynamic pricing

Pricing is computed using the output of the **Best Route Service** (selected route + route metrics).

Pricing factors:

- Distance and ETA (from best route)
- Tolls (from best route)
- Road conditions (if available)
- Delivery type
- Priority
- Weather conditions
- Demand

8.4 Order clubbing (batching)

Goal: reduce cost

- Group orders with same route.
- Group orders with same destination warehouse.
- Prioritize urgent deliveries.

8.5 Warehouse allocation

Goal: select optimal warehouse

- Based on distance, capacity, and load balancing.

9. Database design (MySQL)

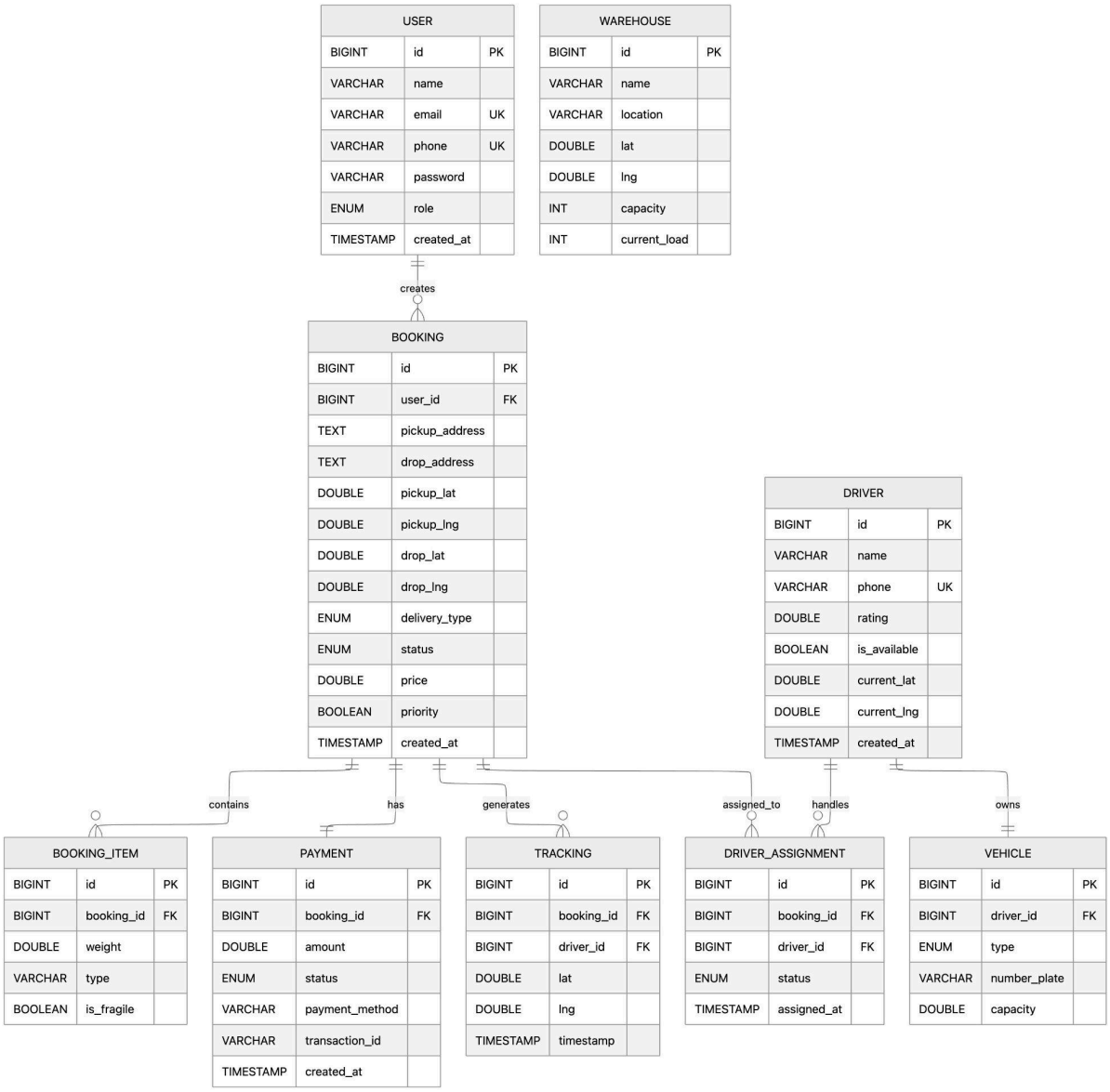
9.1 Final tables

- USER
- DRIVER
- VEHICLE

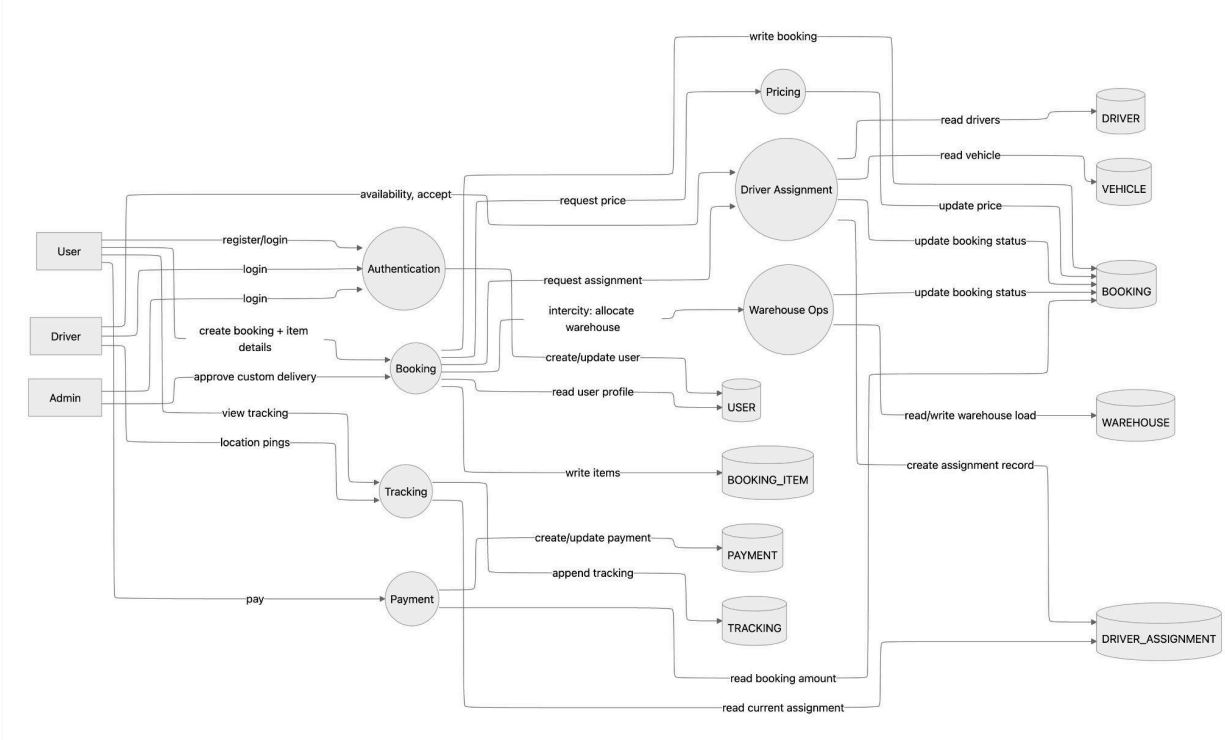
- BOOKING
- BOOKING_ITEM
- WAREHOUSE
- DRIVER_ASSIGNMENT
- PAYMENT
- TRACKING

9.2 Table definitions (DDL)

9.2.1 ER diagram (RDBMS relationships)



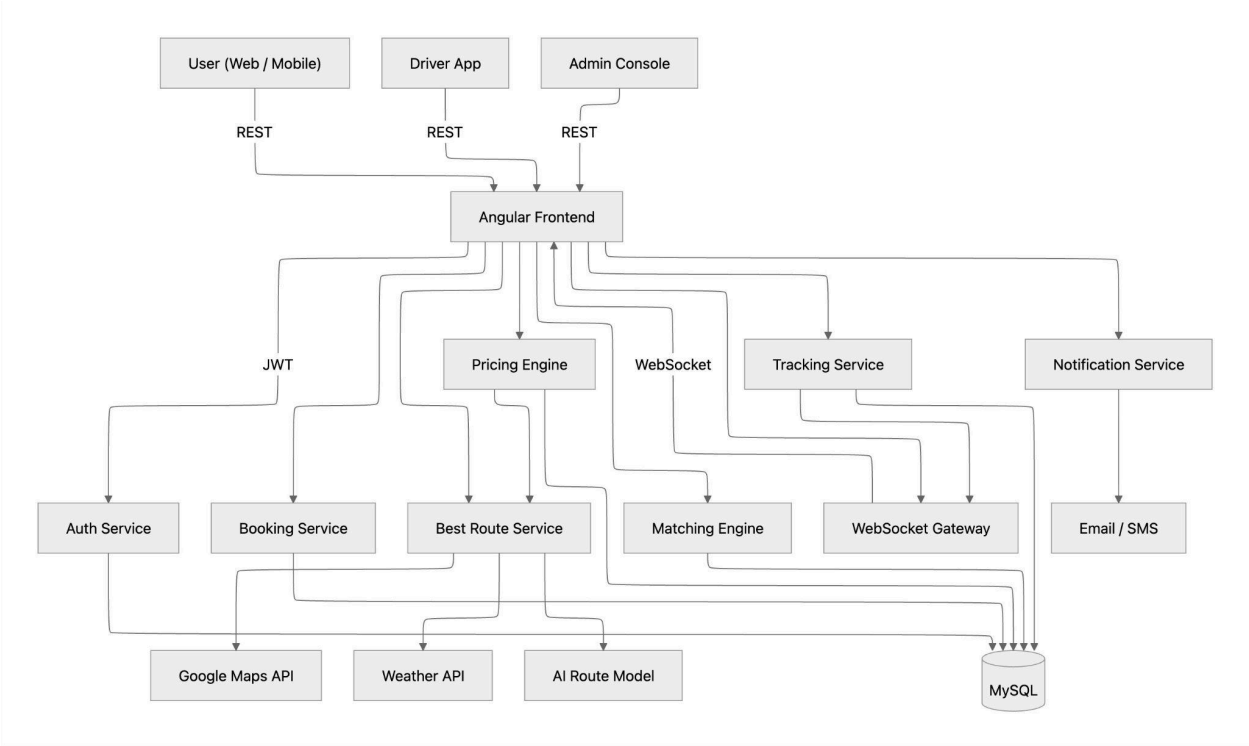
9.2.2 Data flow diagram (DB-centric)



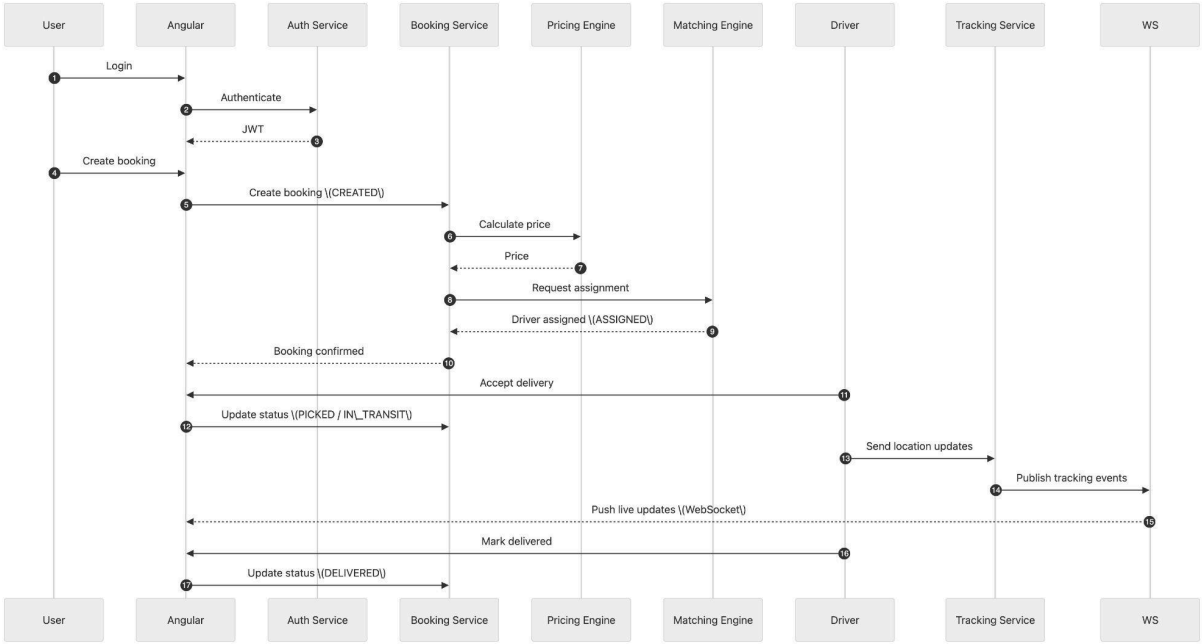
9.3 Relationship summary

- USER 1 → * BOOKING
- BOOKING 1 → * BOOKING_ITEM
- BOOKING 1 → 1 PAYMENT
- BOOKING 1 → * TRACKING
- BOOKING 1 → * DRIVER_ASSIGNMENT
- DRIVER 1 → 1 VEHICLE
- DRIVER 1 → * DRIVER_ASSIGNMENT

10. Architecture diagram (flow)



11. Booking flow diagram



12. Security and access control

- JWT-based authentication.
- Role-based access control (User, Driver, Admin).
- API authorization per endpoint (e.g., Admin-only approve custom deliveries).